

Viterbi-Kalman-RTS algorithm: implementation flow

Mapped-elevation/AltAz serial MultiView delay-gradient solver

Inputs and preprocessing

Rows: t_i , source $_i$, total delay y_i
Geometry: $x_{el,i}$, Δaz_i
Variance: R_i (weight + separation)
Ambiguity spacing: $\Delta = 1/f_{IF}$
Invalid input rows are skipped upstream

Observation and state

$x_i = [g_{el}, g_{az}, \dot{g}_{el}, \dot{g}_{az}]^T$
 $H_i = [x_{el,i}, \Delta az_i, 0, 0]$
 $y_i + n_i \Delta = H_i x_i + e_i$
 $e_i \sim \mathcal{N}(0, R_i)$

No intercept: primary calibrator is zero

Continuous dynamics

$x_i^- = F(\Delta t)x_{i-1}^+$
 $P_i^- = F P_{i-1}^+ F^T + Q$

 F : constant gradient-rate transition
 $Q(q, |\Delta t|)$: local-linear process noise

 q = kalman_factor controls how quickly the gradients may evolve

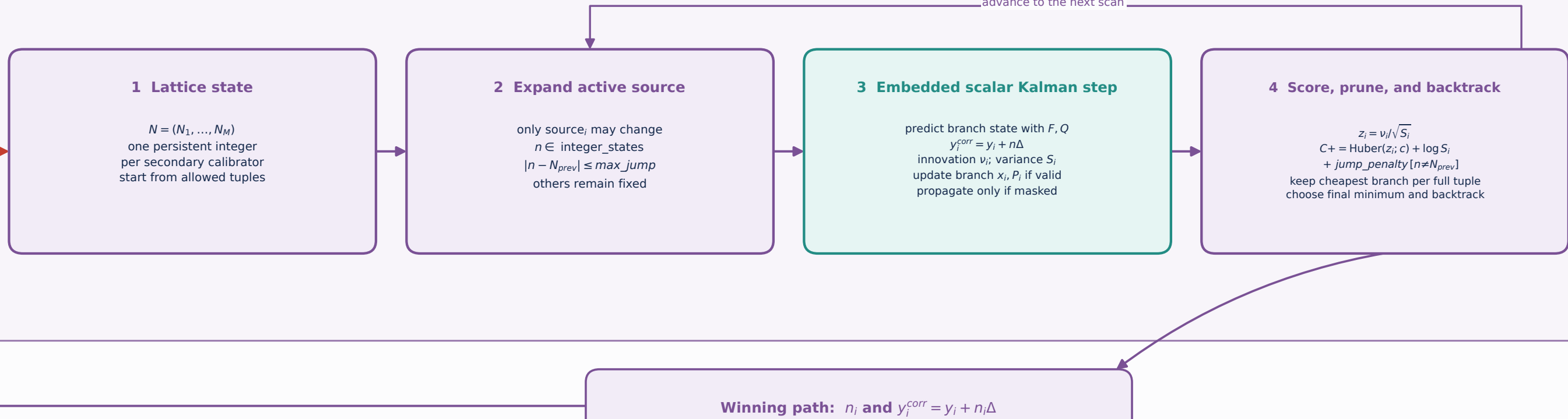
Control parameters

integer_states; fixed initial tuple/map
max_jump; jump_penalty; huber_c
z_out; maximum outlier iterations
rts_smoothing; kalman_factor

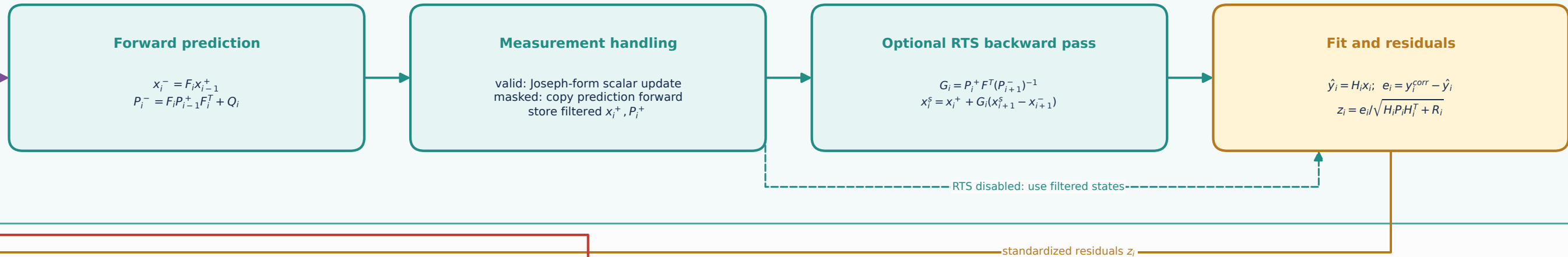
Initialize robust fit

$valid_i = True$ for all rows; $outlier_i = False$; iteration = 0

A. Viterbi dynamic program with a Kalman posterior carried by every branch



B. Continuous-state fit for the selected ambiguity-corrected delay series



C. Persistent outlier mask, stopping rule, and final returned fit

